

TUGAS PRAKTIKUM 5
“SINKRONISASI SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Ddisusun oleh :

Nama : Fajar Satria Nusaputra

NIM : 2410651058

KELAS : KAMIS C

UNIVERSITAS MUHAMMADIYAH JEMBER
FAKULTAS TEKNIK
PRODI TEKNIK INFORMATIKA
2025-2026

DAFTAR ISI

TUGAS PRAKTIKUM 5 “SINKRONISASI SISTEM OPRASI”	1
DAFTAR ISI	2
BAB 1 PROGRAM 1 DEADLOCK VERSION.....	3
BAB 2 PROGRAM 2 RESOURCE	6
BAB 3 PROGRAM 3 LIMITING PHILOSOPHERS	9
BAB 4 TABEL EFISIENSI/FAIRNESS	12

BAB 1

PROGRAM 1 DEADLOCK VERSION

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama nano dining_deadlock.c dengan isi kode ada pada gambar dibawah ini, lalu compile dan jalankan program sehingga hasilnya nampak seperti berikut:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdarg.h>
#define N 3

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
volatile int running = 1;
int eat_count[N];

void msleep(long ms) { usleep(ms * 1000); }

void print_ts(const char *fmt, ...) {
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg) {
    int id = (int)arg;
    free(arg);

    srand((unsigned int)time(NULL) + id * 13);

    while (running) {

        print_ts("🧠 Filsuf-%d sedang berpikir...\n", id+1);
        msleep(100 + rand()%900);

        print_ts("🍴 Filsuf-%d lapar, mencoba ambil sumpit kiri dulu...\n", id+1);

        int left = id;
        int right = (id + 1) % N;

        pthread_mutex_lock(&forks[left]);
        print_ts("// Filsuf-%d mengambil sumpit kiri (%d)\n", id+1, left);
        msleep(50);

        pthread_mutex_lock(&forks[right]);
        print_ts("// Filsuf-%d mengambil sumpit kanan (%d)\n", id+1, right);
```

```

        print_ts("🍴 Filsuf-%d sedang MAKAN...\n", id+1);
        eat_count[id]++;
        msleep(100 + rand()%300);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        print_ts("// Filsuf-%d menaruh kedua sumpit\n", id+1);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv) {
    int run_seconds = 60;
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i] = 0;
    }

    print_ts("=== DINING PHILOSOPHERS: DEADLOCK VERSION ===\n");
    for (int i = 0; i < N; i++) {
        int *id = malloc(sizeof(int));
        if (!id) { perror("malloc"); exit(1); }
        *id = i;
        if (pthread_create(&th[i], NULL, philosopher, id) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    sleep(run_seconds);
    running = 0;

    for (int i = 0; i < N; i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    int total = 0;
    for (int i = 0; i < N; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    printf("Total makan: %d\n", total);

    for (int i = 0; i < N; i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

Hasil

```
koch@Kchnng:~$ ./dining_resource_ordering 1
[02:40:59] === DINING PHILOSOPHERS: RESOURCE ORDERING ===
[02:40:59] 🤔 Filsuf-1 sedang berpikir...
[02:40:59] 🤔 Filsuf-2 sedang berpikir...
[02:40:59] 🤔 Filsuf-3 sedang berpikir...
[02:40:59] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[02:40:59] // Filsuf-3 mengambil sumpit (0)
[02:40:59] // Filsuf-3 mengambil sumpit (2)
[02:40:59] 🍽️ Filsuf-3 sedang MAKAN...
[02:40:59] // Filsuf-3 menaruh kedua sumpit
[02:40:59] 🤔 Filsuf-3 sedang berpikir...
[02:40:59] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[02:40:59] // Filsuf-2 mengambil sumpit (1)
[02:40:59] // Filsuf-2 mengambil sumpit (2)
[02:40:59] 🍽️ Filsuf-2 sedang MAKAN...
[02:40:59] // Filsuf-2 menaruh kedua sumpit
[02:40:59] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[02:40:59] // Filsuf-1 mengambil sumpit (0)
[02:40:59] // Filsuf-1 mengambil sumpit (1)
[02:40:59] 🍽️ Filsuf-1 sedang MAKAN...
[02:40:59] 🤔 Filsuf-2 sedang berpikir...
[02:41:00] // Filsuf-1 menaruh kedua sumpit
[02:41:00] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[02:41:00] // Filsuf-3 mengambil sumpit (0)
[02:41:00] // Filsuf-3 mengambil sumpit (2)
[02:41:00] 🍽️ Filsuf-3 sedang MAKAN...
[02:41:00] // Filsuf-3 menaruh kedua sumpit
[02:41:00] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[02:41:00] // Filsuf-2 mengambil sumpit (1)
[02:41:01] // Filsuf-2 mengambil sumpit (2)
[02:41:01] 🍽️ Filsuf-2 sedang MAKAN...
[02:41:01] // Filsuf-2 menaruh kedua sumpit
[02:41:01]
=== STATISTIK (1 detik) ===
Filsuf-1: Makan 1 kali
Filsuf-2: Makan 2 kali
Filsuf-3: Makan 2 kali
Total makan: 5
koch@Kchnng:~$ |
```

Nah jadi program ini mensimulasikan masalah dining philosophers dengan metode resource ordering, yaitu setiap filsuf mengambil sumpit berdasarkan urutan nomor tertentu untuk mencegah deadlock. Dengan mutex tiap sumpit hanya bisa digunakan satu filsuf pada satu waktu. Hasilnya, semua filsuf dapat berpikir dan makan secara bergantian tanpa saling menunggu selamanya.

BAB 2

PROGRAM 2 RESOURCE

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama `dining_resource_ordering.c` dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 1 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdarg.h>

#define N 3

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
volatile int running = 1;
int eat_count[N];

void msleep(long ms){ usleep(ms * 1000); }

void print_ts(const char *fmt, ...){
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg){
    int id = (int)arg;
    free(arg);

    srand(time(NULL) + id * 17);

    while (running) {
        print_ts("🧠 Filsuf-%d sedang berpikir...\n", id+1);
        msleep(100 + rand()%900);

        print_ts("🍴 Filsuf-%d lapar, mencoba ambil sumpit...\n", id+1);

        int left = id;
        int right = (id + 1) % N;

        // pick in ordered manner: smaller index first
        int first = left < right ? left : right;
        int second = left < right ? right : left;

        pthread_mutex_lock(&forks[first]);
        print_ts("// Filsuf-%d mengambil sumpit (%d)\n", id+1, first);
```

```

        print_ts(" // Filsuf-%d mengambil sumpit (%d)\n", id+1, first);
        msleep(30);
        pthread_mutex_lock(&forks[second]);
        print_ts(" // Filsuf-%d mengambil sumpit (%d)\n", id+1, second);

        print_ts(" 🍴 Filsuf-%d sedang MAKAN...\n", id+1);
        eat_count[id]++;
        msleep(100 + rand()%300);

        pthread_mutex_unlock(&forks[second]);
        pthread_mutex_unlock(&forks[first]);
        print_ts(" // Filsuf-%d menaruh kedua sumpit\n", id+1);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv){
    int run_seconds = 60;
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i=0;i<N;i++){
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i]=0;
    }

    print_ts("=== DINING PHILOSOPHERS: RESOURCE ORDERING ===\n");
    for (int i=0;i<N;i++){
        int *id = malloc(sizeof(int)); *id = i;
        pthread_create(&th[i], NULL, philosopher, id);
    }

    sleep(run_seconds);
    running = 0;

    for (int i=0;i<N;i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    int total=0;
    for (int i=0;i<N;i++){
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    printf("Total makan: %d\n", total);

    for (int i=0;i<N;i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

Hasilnya:

```
koch@Kchng:~$ ./dining_resource_ordering 1
[02:58:59] === DINING PHILOSOPHERS: RESOURCE ORDERING ===
[02:58:59] 🤔 Filsuf-1 sedang berpikir...
[02:58:59] 🤔 Filsuf-2 sedang berpikir...
[02:58:59] 🤔 Filsuf-3 sedang berpikir...
[02:58:59] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[02:58:59] // Filsuf-2 mengambil sumpit (1)
[02:58:59] // Filsuf-2 mengambil sumpit (2)
[02:58:59] 🍽️ Filsuf-2 sedang MAKAN...
[02:58:59] // Filsuf-2 menaruh kedua sumpit
[02:58:59] 🤔 Filsuf-2 sedang berpikir...
[02:59:00] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[02:59:00] // Filsuf-3 mengambil sumpit (0)
[02:59:00] // Filsuf-3 mengambil sumpit (2)
[02:59:00] 🍽️ Filsuf-3 sedang MAKAN...
[02:59:00] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[02:59:00] // Filsuf-3 menaruh kedua sumpit
[02:59:00] // Filsuf-1 mengambil sumpit (0)
[02:59:00] // Filsuf-1 mengambil sumpit (1)
[02:59:00] 🍽️ Filsuf-1 sedang MAKAN...
[02:59:00] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[02:59:01] // Filsuf-1 menaruh kedua sumpit
[02:59:01] // Filsuf-2 mengambil sumpit (1)
[02:59:01] // Filsuf-2 mengambil sumpit (2)
[02:59:01] 🍽️ Filsuf-2 sedang MAKAN...
[02:59:01] // Filsuf-2 menaruh kedua sumpit
[02:59:01]
=== STATISTIK (1 detik) ===
Filsuf-1: Makan 1 kali
Filsuf-2: Makan 2 kali
Filsuf-3: Makan 1 kali
Total makan: 4
koch@Kchng:~$ |
```

Bisa kita lihat berdasarkan gambar diatas yang menggunakan solusi Resource Ordering, setiap filsuf mengambil sumpit dalam urutan yang sama berdasarkan nomor terkecil terlebih dahulu. Dengan cara ini, tidak akan terbentuk siklus saling tunggu sehingga deadlock dapat dihindari sepenuhnya. Program berjalan stabil dan semua filsuf dapat makan bergantian tanpa macet.

BAB 3

PROGRAM 3 LIMITING PHILOSOPHERS

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama dining_limited.c dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 3 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <stdarg.h>

#define N 3

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
volatile int running = 1;
int eat_count[N];
sem_t room; // limits how many philosophers can attempt to pick forks

void msleep(long ms){ usleep(ms * 1000); }

void print_ts(const char *fmt, ...){
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg){
    int id = (int)arg;
    free(arg);

    srand(time(NULL) + id * 19);

    while (running) {
        print_ts("🧠 Filsuf-%d sedang berpikir...\n", id+1);
        msleep(100 + rand()%900);

        print_ts("🍴 Filsuf-%d lapar, menunggu kesempatan masuk meja (semaphore)...\n", id+1);

        // limit number of philosophers that can try to pick up forks
        sem_wait(&room);

        int left = id;
```

```

        int right = (id + 1) % N;

        // now safely pick forks (no guaranteed order needed because room size prevents deadlock)
        pthread_mutex_lock(&forks[left]);
        print_ts(" // Filsuf-%d mengambil sumpit kiri (%d)\n", id+1, left);
        msleep(30);
        pthread_mutex_lock(&forks[right]);
        print_ts(" // Filsuf-%d mengambil sumpit kanan (%d)\n", id+1, right);

        print_ts(" 🍴 Filsuf-%d sedang MAKAN...\n", id+1);
        eat_count[id]++;
        msleep(100 + rand()%300);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        print_ts(" // Filsuf-%d menaruh kedua sumpit\n", id+1);

        // leave room so another philosopher can enter
        sem_post(&room);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv){
    int run_seconds = 60;
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i=0;i<N;i++){
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i]=0;
    }

    // Allow at most N-1 philosophers to try to pick forks at once (prevent cycle)
    sem_init(&room, 0, N-1);

    print_ts("=== DINING PHILOSOPHERS: LIMITING PHILOSOPHERS (max %d) ===\n", N-1);
    for (int i=0;i<N;i++){
        int *id = malloc(sizeof(int)); *id = i;
        pthread_create(&th[i], NULL, philosopher, id);
    }

    sleep(run_seconds);
    running = 0;

    for (int i=0;i<N;i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    int total=0;
    for (int i=0;i<N;i++){
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    printf("Total makan: %d\n", total);

    sem_destroy(&room);
    for (int i=0;i<N;i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

Hasil:

```
[03:05:45] 🍴 Filsuf-2 sedang MAKAN...
[03:05:45] 🤔 Filsuf-3 sedang berpikir...
[03:05:45] 🕒 Filsuf-3 lapar, menunggu kesempatan masuk meja (semaphore)...
[03:05:45] // Filsuf-2 menaruh kedua sumpit
[03:05:45] // Filsuf-3 mengambil sumpit kiri (2)
[03:05:45] 🕒 Filsuf-1 lapar, menunggu kesempatan masuk meja (semaphore)...
[03:05:45] // Filsuf-1 mengambil sumpit kiri (0)
[03:05:45] // Filsuf-1 mengambil sumpit kanan (1)
[03:05:45] 🍴 Filsuf-1 sedang MAKAN...
[03:05:45] // Filsuf-1 menaruh kedua sumpit
[03:05:45] // Filsuf-3 mengambil sumpit kanan (0)
[03:05:45] 🍴 Filsuf-3 sedang MAKAN...
[03:05:46] // Filsuf-3 menaruh kedua sumpit
[03:05:46]
=== STATISTIK (3 detik) ===
Filsuf-1: Makan 4 kali
Filsuf-2: Makan 3 kali
Filsuf-3: Makan 4 kali
Total makan: 11
koch@Kchn:~$ |
```

Disini outputnya menampilkan adanya menunggu “masuk meja” yang artinya mekanisme semaphore sedang berkerja untuk membatasi jumlah filsuf. Tatistik terakhir menunjukkan jumlah makan yang merata antara filsuf, nah ini efektif mencegah dan membuat adil agar semua filsuf semua makan tanpa ada yang kelaparan

BAB 4

TABEL EFISIENSI/FAIRNESS

Solusi \ Filsuf	F1 F2 F3 Total
ResourceOrdering	1 2 1 4
Limiting	4 3 4 11

Berdasarkan tabel diatas, dari kedua pendekatan yang diuji, solusi “Limiting Philosophers” merupakan metode yang paling adil (fair) karena semua filsuf mendapatkan kesempatan untuk makan secara bergantian tanpa mengalami kelaparan (starvation). sementara pada solusi Resource Ordering, walaupun deadlock dapat dicegah, terdapat kemungkinan beberapa filsuf terus kalah cepat mengambil sumpit dan makan lebih sedikit.